# Resume RAG and Job Matcher

## Objective

This project implements a retrieval-augmented resume search and job-matching pipeline. It processes resume files, extracts candidate metadata, indexes resume chunks using vector embeddings, and ranks candidates against job descriptions.

The system combines semantic retrieval with lexical matching and applies explicit eligibility rules for must-have skills and minimum experience.

## Dataset

- 30 synthetic resumes
- 5 synthetic job descriptions
- Supported resume formats: TXT, PDF, and DOCX
- Synthetic candidate profiles are used for educational testing; they do not represent real applicants.

## Main components

1. File processing and text extraction
2. Metadata extraction and section-preserving chunking
3. Embedding generation and ChromaDB indexing
4. Hybrid retrieval and candidate scoring
5. Eligibility filtering, evaluation, and error-case testing

Drive: https://drive.google.com/drive/folders/1dNheyWQJ3_nKuiGa6OTnE-0WY979gMVX?usp=sharing

```
%pip install -q chromadb sentence-transformers pypdf python-docx scikit-learn pandas matplotlib
```

```
from google.colab import drive
from pathlib import Path

drive.mount("/content/drive")
PROJECT_DIR = Path("/content/drive/MyDrive/ResumeRAG")

RESUME_DIR = PROJECT_DIR / "resumes"
JD_DIR = PROJECT_DIR / "job_descriptions"
DB_DIR = PROJECT_DIR / "chroma_db"
RESULTS_DIR = PROJECT_DIR / "results"

for directory in [RESUME_DIR, JD_DIR, DB_DIR, RESULTS_DIR]:
    directory.mkdir(parents=True, exist_ok=True)

print("Google Drive connected.")
print("Project directory:", PROJECT_DIR)
print("Resumes directory:", RESUME_DIR)
print("Job descriptions directory:", JD_DIR)
print("ChromaDB directory:", DB_DIR)
print("Results directory:", RESULTS_DIR)
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force
Google Drive connected.
Project directory: /content/drive/MyDrive/ResumeRAG
Resumes directory: /content/drive/MyDrive/ResumeRAG/resumes
Job descriptions directory: /content/drive/MyDrive/ResumeRAG/job_descriptions
ChromaDB directory: /content/drive/MyDrive/ResumeRAG/chroma_db
Results directory: /content/drive/MyDrive/ResumeRAG/results
```

```
import sys

if str(PROJECT_DIR) not in sys.path:
    sys.path.insert(0, str(PROJECT_DIR))

import fs_tools

listed_resumes = fs_tools.list_files(str(RESUME_DIR))
print("Files listed:", len(listed_resumes))

if not listed_resumes:
```

```
    raise AssertionError("fs_tools.list_files returned no resumes.")

sample_path = listed_resumes[0]["path"]
read_result = fs_tools.read_file(sample_path)

print("Sample file:", listed_resumes[0]["filename"])
print("Read succeeded:", read_result["success"])
print("Characters extracted:", read_result["metadata"]["character_count"])

assert read_result["success"], read_result["error"]
assert read_result["content"].strip(), "Extracted content is empty."
assert len(listed_resumes) >= 30, "Expected at least 30 resumes."

print("\nfs_tools integration prerequisites passed.")
```

```
Files listed: 30
Sample file: candidate_01.txt
Read succeeded: True
Characters extracted: 804

fs_tools integration prerequisites passed.
```

## 1. Environment and Dataset Preparation

Google Drive stores the project files, resume dataset, job descriptions, vector database, and generated results.

The `fs_tools.py` utility provides file listing, reading, writing, and keyword-search functionality. Synthetic data is used to make the pipeline reproducible without exposing real candidate information.

The dataset-generation cell should be safe to rerun and should leave at least 30 resumes and 5 job descriptions available for processing.

```
fs_tools_code = r'''
from pathlib import Path
from datetime import datetime

from pypdf import PdfReader
from docx import Document

SUPPORTED_EXTENSIONS = {".pdf", ".txt", ".docx"}


def _file_metadata(path: Path) -> dict:
    stat = path.stat()
    return {
        "filename": path.name,
        "extension": path.suffix.lower(),
        "size_bytes": stat.st_size,
        "modified_time": datetime.fromtimestamp(
            stat.st_mtime
        ).isoformat(timespec="seconds"),
    }


def _extract_text(path: Path) -> str:
    extension = path.suffix.lower()

    if extension == ".txt":
        try:
            return path.read_text(encoding="utf-8-sig")
        except UnicodeDecodeError:
            return path.read_text(encoding="latin-1")

    if extension == ".pdf":
        reader = PdfReader(str(path))
        return "\n".join(
            page.extract_text() or "" for page in reader.pages
        )

    if extension == ".docx":
        document = Document(path)
        paragraphs = [p.text for p in document.paragraphs]
        return "\n".join(paragraphs)

    raise ValueError(f"Unsupported file extension: {extension}")


def read_file(filepath: str) -> dict:
    path = Path(filepath).expanduser()
```

```
        try:
            if not path.exists():
                raise FileNotFoundError(f"File not found: {filepath}")
            if not path.is_file():
                raise ValueError(f"Path is not a file: {filepath}")

            content = _extract_text(path)

            return {
                "success": True,
                "filepath": str(path.resolve()),
                "content": content,
                "metadata": {
                    **_file_metadata(path),
                    "character_count": len(content),
                    "page_count": (
                        len(PdfReader(str(path)).pages)
                        if path.suffix.lower() == ".pdf"
                        else None
                    ),
                },
                "error": None,
            }
        except Exception as exc:
            return {
                "success": False,
                "filepath": str(path),
                "content": None,
                "metadata": None,
                "error": f"{type(exc).__name__}: {exc}",
            }


    def list_files(directory: str, extension: str = None) -> list:
        path = Path(directory).expanduser()
        if not path.is_dir():
            return []

        normalized_extension = None
        if extension:
            normalized_extension = extension.lower()
            if not normalized_extension.startswith("."):
                normalized_extension = "." + normalized_extension

        results = []
        try:
            for item in sorted(path.iterdir(), key=lambda p: p.name.lower()):
                if not item.is_file():
                    continue
                if (
                    normalized_extension is not None
                    and item.suffix.lower() != normalized_extension
                ):
                    continue
                try:
                    results.append({
                        **_file_metadata(item),
                        "path": str(item.resolve()),
                    })
                except OSError:
                    continue
        except OSError:
            return []

        return results


    def write_file(filepath: str, content: str) -> dict:
        path = Path(filepath).expanduser()
        try:
            if not isinstance(content, str):
                raise TypeError("content must be a string")
            path.parent.mkdir(parents=True, exist_ok=True)
            path.write_text(content, encoding="utf-8")
            return {
                "success": True,
                "filepath": str(path.resolve()),
                "bytes_written": path.stat().st_size,
                "error": None,
            }
        except Exception as exc:
```

```
        return {
            "success": False,
            "filepath": str(path),
            "bytes_written": 0,
            "error": f"{type(exc).__name__}: {exc}",
        }


def search_in_file(filepath: str, keyword: str) -> dict:
    if not isinstance(keyword, str) or not keyword.strip():
        return {
            "success": False, "filepath": str(filepath),
            "keyword": keyword, "match_count": 0, "matches": [],
            "error": "keyword must be a non-empty string",
        }

    result = read_file(filepath)
    if not result["success"]:
        return {
            "success": False, "filepath": str(filepath),
            "keyword": keyword, "match_count": 0, "matches": [],
            "error": result["error"],
        }

    content = result["content"]
    term = keyword.strip()
    matches = []
    start = 0

    while True:
        index = content.lower().find(term.lower(), start)
        if index == -1:
            break

        matches.append({
            "match": content[index:index + len(term)],
            "position": index,
            "context": content[
                max(0, index - 80):
                min(len(content), index + len(term) + 80)
            ],
        })
        start = index + len(term)

    return {
        "success": True, "filepath": result["filepath"],
        "keyword": term, "match_count": len(matches),
        "matches": matches, "error": None,
    }
'''

fs_tools_path = PROJECT_DIR / "fs_tools.py"
fs_tools_path.write_text(fs_tools_code.strip() + "\n", encoding="utf-8")

print("Created:", fs_tools_path)
print("File exists:", fs_tools_path.exists())
print("File size:", fs_tools_path.stat().st_size, "bytes")
```

```
Created: /content/drive/MyDrive/ResumeRAG/fs_tools.py
File exists: True
File size: 5070 bytes
```

```
from pathlib import Path

import random

RESUME_DIR.mkdir(parents=True, exist_ok=True)
JD_DIR.mkdir(parents=True, exist_ok=True)

profiles = [
    {
        "role": "Python Backend Engineer",
        "skills": "Python, Django, REST API, PostgreSQL, Docker",
        "education": "B.Tech in Computer Science"
    },
    {
        "role": "Java Developer",
        "skills": "Java, Spring Boot, SQL, Docker, Git",
        "education": "B.Tech in Information Technology"
    },
```

```
    {
        "role": "Machine Learning Engineer",
        "skills": "Python, Machine Learning, PyTorch, Pandas, NumPy",
        "education": "M.Tech in Artificial Intelligence"
    },
    {
        "role": "Data Scientist",
        "skills": "Python, SQL, Scikit-learn, Pandas, Data Analysis",
        "education": "M.Sc. in Data Science"
    },
    {
        "role": "Cloud Engineer",
        "skills": "AWS, Terraform, Docker, Kubernetes, Linux",
        "education": "B.Tech in Computer Science"
    },
    {
        "role": "Frontend Developer",
        "skills": "JavaScript, TypeScript, React, REST API, Git",
        "education": "B.Tech in Information Technology"
    },
    {
        "role": "Cybersecurity Engineer",
        "skills": "Cybersecurity, Linux, Python, Networking, AWS",
        "education": "B.Tech in Cybersecurity"
    },
    {
        "role": "NLP Engineer",
        "skills": "Python, NLP, LLM, PyTorch, Generative AI",
        "education": "M.Tech in Artificial Intelligence"
    },
    {
        "role": "DevOps Engineer",
        "skills": "Linux, Docker, Kubernetes, CI/CD, AWS",
        "education": "B.Tech in Computer Science"
    },
    {
        "role": "Data Engineer",
        "skills": "Python, SQL, Spark, Kafka, PostgreSQL",
        "education": "B.Tech in Computer Science"
    }
]

first_names = [
    "Alex", "Jordan", "Taylor", "Morgan", "Riley",
    "Casey", "Avery", "Jamie", "Cameron", "Drew"
]

last_names = [
    "Sharma", "Patel", "Singh", "Gupta", "Kumar",
    "Mehta", "Rao", "Verma", "Das", "Joshi"
]

existing_resumes = list(RESUME_DIR.glob("*.txt"))

if len(existing_resumes) < 30:
    for i in range(30):
        profile = profiles[i % len(profiles)]
        years = 3 + (i % 6)
        name = f"{first_names[i % 10]} {last_names[(i // 3) % 10]}"

        resume_text = f"""
SYNTHETIC RESUME – EDUCATIONAL USE ONLY

{name}
Target Profile: {profile['role']}

SUMMARY
Technology professional with {years} years of experience.
Experienced in software development, troubleshooting,
collaboration, and delivering technical solutions.

TECHNICAL SKILLS
{profile['skills']}

EXPERIENCE
Software Engineer | Example Technology Company

Professional experience: {years} years.
Responsibilities include developing solutions, debugging
issues, collaborating with teams, and maintaining systems.

EDUCATION
```

```
{profile['education']}, Example University
""".strip()

        resume_path = RESUME_DIR / f"candidate_{i + 1:02d}.txt"
        resume_path.write_text(resume_text, encoding="utf-8")


job_descriptions = {
    "python_backend.txt": """
JOB TITLE: Python Backend Engineer

RESPONSIBILITIES
Build and maintain backend services, APIs, and databases.

REQUIRED SKILLS
Python, Django, REST API, PostgreSQL.

MUST-HAVE: Python, Django.

MINIMUM EXPERIENCE: 3 years.
""",
    "java_backend.txt": """
JOB TITLE: Java Backend Developer

RESPONSIBILITIES
Develop scalable backend applications and REST services.

REQUIRED SKILLS
Java, Spring Boot, SQL, Docker, Git.

MUST-HAVE: Java, Spring Boot.

MINIMUM EXPERIENCE: 3 years.
""",
    "machine_learning.txt": """
JOB TITLE: Machine Learning Engineer

RESPONSIBILITIES
Develop, evaluate, and deploy machine learning models.

REQUIRED SKILLS
Python, Machine Learning, PyTorch, Pandas, NumPy.

MUST-HAVE: Python, Machine Learning.

MINIMUM EXPERIENCE: 2 years.
""",
    "cloud_devops.txt": """
JOB TITLE: Cloud DevOps Engineer

RESPONSIBILITIES
Manage cloud infrastructure, containers, and deployments.

REQUIRED SKILLS
AWS, Terraform, Docker, Kubernetes, Linux, CI/CD.

MUST-HAVE: AWS, Docker.

MINIMUM EXPERIENCE: 2 years.
""",
    "data_engineer.txt": """
JOB TITLE: Data Engineer

RESPONSIBILITIES
Build data pipelines and manage data infrastructure.

REQUIRED SKILLS
Python, SQL, Spark, Kafka, PostgreSQL.

MUST-HAVE: Python, SQL.

MINIMUM EXPERIENCE: 2 years.
"""
}

for filename, content in job_descriptions.items():
    jd_path = JD_DIR / filename

    if not jd_path.exists():
        jd_path.write_text(content.strip(), encoding="utf-8")

resume_files = sorted(RESUME_DIR.glob("*.txt"))
```

```
jd_files = sorted(JD_DIR.glob("*.txt"))

print("DATASET VALIDATION")
print("-" * 40)
print(f"Resume count: {len(resume_files)}")
print(f"Job description count: {len(jd_files)}")
print("Resume directory:", RESUME_DIR)
print("Job description directory:", JD_DIR)

assert len(resume_files) >= 30, "Expected at least 30 TXT resumes."
assert len(jd_files) >= 5, "Expected at least 5 TXT job descriptions."

print("\nDataset validation passed.")
```

```
DATASET VALIDATION
----------------------------------------
Resume count: 30
Job description count: 5
Resume directory: /content/drive/MyDrive/ResumeRAG/resumes
Job description directory: /content/drive/MyDrive/ResumeRAG/job_descriptions

Dataset validation passed.
```

## 2. Text Processing and Resume Chunking

Resume text is extracted through the shared file utility. Metadata extraction identifies candidate names, technical skills, experience, and education.

The chunking function identifies common resume section headers and divides long sections into smaller text chunks. Each chunk retains its section name so that retrieval results can be associated with the original resume structure.

The default configuration uses a chunk size of 800 characters and an overlap of 120 characters. These values are measured in characters, not tokens.

```
import re
import json
import time
import math
import hashlib
from pathlib import Path
from collections import Counter, defaultdict

import chromadb
import pandas as pd
import matplotlib.pyplot as plt

from sentence_transformers import SentenceTransformer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from pypdf import PdfReader
from docx import Document



from fs_tools import read_file, list_files


def extract_text(file_path):
    """Extract text through the shared fs_tools utility."""
    result = read_file(str(file_path))

    if not result["success"]:
        print(
            f"Could not read {Path(file_path).name}: "
            f"{result['error']}"
        )
        return ""

    return result["content"]


sample_resume = sorted(RESUME_DIR.glob("*.txt"))[0]
sample_text = extract_text(sample_resume)

print("Sample file:", sample_resume.name)
print("Extracted characters:", len(sample_text))
print("\nText preview:\n")
print(sample_text[:500])
```

```
assert len(sample_text.strip()) > 0, "Text extraction failed."
print("\nText extraction test passed.")
```

```
Sample file: candidate_01.txt
Extracted characters: 804

Text preview:

SYNTHETIC RESUME — EDUCATIONAL USE ONLY

Alex Sharma
Target Profile: Python Backend Engineer

SUMMARY
Technology professional with 3 years of experience.
Interested in software engineering and solving technical problems.

TECHNICAL SKILLS
Python, Django, REST API, PostgreSQL, Docker

EXPERIENCE
Software Engineer | Example Technology Company

Professional experience: 3 years.
Developed and maintained solutions related to python backend engineer.
Worked with Python, Django, REST API.
Collaborated

Text extraction test passed.
```

```
SKILL_VOCABULARY = [
    "Python", "Java", "C++", "C", "Golang", "JavaScript",
    "TypeScript", "React", "Django", "Spring Boot", "REST API",
    "PostgreSQL", "SQL", "MySQL", "MongoDB", "Docker",
    "Kubernetes", "AWS", "Azure", "Google Cloud", "Terraform",
    "Linux", "Networking", "Cybersecurity", "Machine Learning",
    "Deep Learning", "PyTorch", "TensorFlow", "Pandas", "NumPy",
    "Scikit-learn", "Data Analysis", "NLP", "LLM",
    "Generative AI", "Spark", "Kafka", "CI/CD", "Git",
    "Data Engineering"
]


def extract_metadata(text, filename=""):
    """Extract basic candidate metadata from resume text."""
    lines = [line.strip() for line in text.splitlines() if line.strip()]

    name = "Unknown"
    for line in lines[:8]:
        if (
            not line.isupper()
            and not line.lower().startswith(("target profile:", "summary", "name:"))
            and "synthetic resume" not in line.lower()
            and len(line.split()) in (2, 3, 4)
            and not any(char.isdigit() for char in line)
        ):
            name = line.removeprefix("Name:").strip()
            break

    found_skills = []
    for skill in SKILL_VOCABULARY:
        pattern = r"(?<!\w)" + re.escape(skill) + r"(?!\w)"
        if re.search(pattern, text, flags=re.IGNORECASE):
            found_skills.append(skill)

    experience_years = None
    experience_patterns = [
        r"(?:professional\s+)?experience\s*:\s*(\d+(?:\.\d+)?)\s*years?",
        r"(\d+(?:\.\d+)?)\+?\s*years?\s+of\s+experience",
        r"experience\s+of\s+(\d+(?:\.\d+)?)\s*years?"
    ]

    for pattern in experience_patterns:
        match = re.search(pattern, text, flags=re.IGNORECASE)
        if match:
            experience_years = float(match.group(1))
            break

    education = "Not specified"
    for line in lines:
        if re.search(
            r"\b(B\.?Tech|M\.?Tech|B\.?E\.?|M\.?E\.?|"
            r"Bachelor|Master|M\.?Sc\.?|B\.?Sc\.?|Ph\.?D)\b",
```

```
                line,
                flags=re.IGNORECASE
            ):
                education = line
                break

        return {
            "name": name,
            "skills": found_skills,
            "experience_years": experience_years,
            "education": education,
            "resume_filename": Path(filename).name if filename else ""
        }

    sample_metadata = extract_metadata(sample_text, sample_resume.name)

    print(json.dumps(sample_metadata, indent=2))

    assert sample_metadata["name"] != "Unknown", "Name extraction failed."
    assert sample_metadata["skills"], "Skill extraction failed."
    assert sample_metadata["experience_years"] is not None, "Experience extraction failed."

    print("\nMetadata extraction test passed.")
```

```
{
  "name": "Alex Sharma",
  "skills": [
    "Python",
    "Django",
    "REST API",
    "PostgreSQL",
    "Docker"
  ],
  "experience_years": 3.0,
  "education": "B.Tech in Computer Science, Example University",
  "resume_filename": "candidate_01.txt"
}

Metadata extraction test passed.
```

```
SECTION_HEADERS = {
    "SUMMARY",
    "PROFILE",
    "OBJECTIVE",
    "TECHNICAL SKILLS",
    "SKILLS",
    "EXPERIENCE",
    "WORK EXPERIENCE",
    "PROFESSIONAL EXPERIENCE",
    "EDUCATION",
    "PROJECTS",
    "CERTIFICATIONS",
    "ACHIEVEMENTS"
}


def chunk_document(text, chunk_size=800, overlap=120):
    """
    Split a document into chunks while preserving section boundaries
    where possible.

    chunk_size and overlap are measured in characters.
    """
    if chunk_size <= 0:
        raise ValueError("chunk_size must be greater than zero.")

    if overlap < 0 or overlap >= chunk_size:
        raise ValueError("overlap must be >= 0 and < chunk_size.")

    text = text.replace("\r\n", "\n").replace("\r", "\n").strip()

    if not text:
        return []

    sections = []
    current_header = "GENERAL"
    current_lines = []

    for line in text.splitlines():
        cleaned = line.strip()
        normalized = re.sub(r"[:\s]+$", "", cleaned).upper()

        if normalized in SECTION_HEADERS:
```

```
            if current_lines:
                sections.append(
                    (current_header, "\n".join(current_lines).strip())
                )

            current_header = normalized
            current_lines = [cleaned]
        else:
            current_lines.append(line)

    if current_lines:
        sections.append(
            (current_header, "\n".join(current_lines).strip())
        )

    chunks = []

    for section_name, section_text in sections:
        if not section_text:
            continue

        if len(section_text) <= chunk_size:
            chunks.append({
                "section": section_name,
                "text": section_text
            })
            continue

        start = 0

        while start < len(section_text):
            end = min(start + chunk_size, len(section_text))

            # Prefer breaking at a newline or space near the chunk limit.
            if end < len(section_text):
                newline_pos = section_text.rfind("\n", start, end)
                space_pos = section_text.rfind(" ", start, end)
                break_pos = max(newline_pos, space_pos)

                if break_pos > start + chunk_size // 2:
                    end = break_pos

            chunk_text = section_text[start:end].strip()

            if chunk_text:
                chunks.append({
                    "section": section_name,
                    "text": chunk_text
                })

            if end >= len(section_text):
                break

            start = max(start + 1, end - overlap)

    return chunks


# Test chunking on a sample resume
sample_chunks = chunk_document(
    sample_text,
    chunk_size=800,
    overlap=120
)

print("Sample resume:", sample_resume.name)
print("Number of chunks:", len(sample_chunks))

for index, chunk in enumerate(sample_chunks[:5]):
    print(f"\n--- Chunk {index + 1} | Section: {chunk['section']} ---")
    print(chunk["text"][:250])

assert sample_chunks, "Chunking returned no chunks."
assert all(chunk["text"].strip() for chunk in sample_chunks)

print("\nChunking test passed.")
```

```
Sample resume: candidate_01.txt
Number of chunks: 7

--- Chunk 1 | Section: GENERAL ---
SYNTHETIC RESUME — EDUCATIONAL USE ONLY
```

```
Alex Sharma
Target Profile: Python Backend Engineer

--- Chunk 2 | Section: SUMMARY ---
SUMMARY
Technology professional with 3 years of experience.
Interested in software engineering and solving technical problems.

--- Chunk 3 | Section: TECHNICAL SKILLS ---
TECHNICAL SKILLS
Python, Django, REST API, PostgreSQL, Docker

--- Chunk 4 | Section: EXPERIENCE ---
EXPERIENCE
Software Engineer | Example Technology Company

Professional experience: 3 years.
Developed and maintained solutions related to python backend engineer.
Worked with Python, Django, REST API.
Collaborated with teams to improve reliability a

--- Chunk 5 | Section: PROJECTS ---
PROJECTS
Built a prototype related to python backend engineer.
Used REST API, PostgreSQL, Docker to develop and test features.

Chunking test passed.
```

## 3. Embeddings and Vector Database

The `ResumeRAG` component uses `sentence-transformers/all-MiniLM-L6-v2` to generate normalized vector embeddings.

Resume chunks and their metadata are stored in a persistent ChromaDB collection using cosine distance. Metadata includes the resume filename, candidate name, skills, experience, education, section, and chunk index.

At query time, the job description is embedded and compared with indexed resume chunks. The most relevant chunks provide evidence for candidate ranking.

```
class ResumeRAG:
    """Retrieval-Augmented Generation infrastructure for resumes."""

    def __init__(
        self,
        db_dir=DB_DIR,
        model_name="sentence-transformers/all-MiniLM-L6-v2"
    ):
        self.db_dir = Path(db_dir)
        self.db_dir.mkdir(parents=True, exist_ok=True)

        print("Loading embedding model...")
        self.model = SentenceTransformer(model_name)
        self.model_name = model_name

        self.client = chromadb.PersistentClient(
            path=str(self.db_dir)
        )

        try:
            self.client.delete_collection("resume_chunks")
        except Exception:
            pass

        self.collection = self.client.create_collection(
            name="resume_chunks",
            metadata={"hnsw:space": "cosine"}
        )

        print("Embedding model loaded.")
        print("ChromaDB collection initialized.")

    def index_resumes(self, resume_dir=RESUME_DIR):
        """Extract, chunk, embed, and index resumes."""
        resume_dir = Path(resume_dir)

        file_records = list_files(str(resume_dir))
        resume_files = [
            Path(record["path"])
            for record in file_records
            if record["extension"] in {".txt", ".pdf", ".docx"}
        ]
```

```
        if not resume_files:
            raise ValueError(
                f"No supported resume files found in {resume_dir}"
            )

        ids = []
        documents = []
        metadatas = []

        for resume_path in resume_files:
            text = extract_text(resume_path)

            if not text.strip():
                print(f"Skipping empty resume: {resume_path.name}")
                continue

            metadata = extract_metadata(text, resume_path.name)
            chunks = chunk_document(text)

            for chunk_index, chunk in enumerate(chunks):
                chunk_id = hashlib.sha256(
                    f"{resume_path.name}::{chunk_index}".encode()
                ).hexdigest()

                ids.append(chunk_id)
                documents.append(chunk["text"])

                metadatas.append({
                    "resume_filename": resume_path.name,
                    "candidate_name": metadata["name"],
                    "skills": ", ".join(metadata["skills"]),
                    "experience_years": (
                        metadata["experience_years"]
                        if metadata["experience_years"] is not None
                        else -1.0
                    ),
                    "education": metadata["education"],
                    "section": chunk["section"],
                    "chunk_index": chunk_index
                })

        if not documents:
            raise ValueError("No resume chunks were created.")

        batch_size = 64

        for start in range(0, len(documents), batch_size):
            end = start + batch_size

            batch_documents = documents[start:end]
            batch_ids = ids[start:end]
            batch_metadatas = metadatas[start:end]

            embeddings = self.model.encode(
                batch_documents,
                normalize_embeddings=True,
                show_progress_bar=False
            ).tolist()

            self.collection.upsert(
                ids=batch_ids,
                documents=batch_documents,
                metadatas=batch_metadatas,
                embeddings=embeddings
            )

        indexed_resumes = len({
            metadata["resume_filename"]
            for metadata in metadatas
        })

        print(f"Resumes indexed: {indexed_resumes}")
        print(f"Chunks indexed: {len(documents)}")

        return {
            "resume_count": indexed_resumes,
            "chunk_count": len(documents)
        }

    def search(self, query, top_k=10):
        """Return the most semantically similar resume chunks."""
        if not query or not query.strip():
```

```
            return []

        total_chunks = self.collection.count()

        if total_chunks == 0:
            raise ValueError("No resume chunks are indexed yet.")

        query_embedding = self.model.encode(
            [query],
            normalize_embeddings=True
        ).tolist()

        result = self.collection.query(
            query_embeddings=query_embedding,
            n_results=min(top_k, total_chunks),
            include=["documents", "metadatas", "distances"]
        )

        hits = []

        for i, document in enumerate(result["documents"][0]):
            metadata = result["metadatas"][0][i]
            distance = result["distances"][0][i]

            hits.append({
                "text": document,
                "metadata": metadata,
                "similarity": max(0.0, min(1.0, 1.0 - distance)),
                "distance": distance
            })

        return hits


print("ResumeRAG class defined successfully.")
```

```
ResumeRAG class defined successfully.
```

```
rag = ResumeRAG(
    db_dir=DB_DIR,
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

index_stats = rag.index_resumes(RESUME_DIR)

indexed_chunks = rag.collection.count()

indexed_files = rag.collection.get(
    include=["metadatas"]
)

unique_resumes = len({
    metadata["resume_filename"]
    for metadata in indexed_files["metadatas"]
})

print("\nINDEXING SUMMARY")
print("-" * 40)
print("Resumes found:", len(list(RESUME_DIR.glob("*.txt"))))
print("Unique resumes indexed:", unique_resumes)
print("Chunks in ChromaDB:", indexed_chunks)

assert unique_resumes >= 30, "Fewer than 30 resumes were indexed."
assert indexed_chunks > 0, "ChromaDB contains no chunks."

print("\nResume indexing and verification passed.")
```

```
Loading embedding model...
Loading weights: 100%  103/103 [00:00<00:00, 1101.74it/s]
Embedding model loaded.
ChromaDB collection initialized.
Resumes indexed: 30
Chunks indexed: 210

INDEXING SUMMARY
----------------------------------------
Resumes found: 30
Unique resumes indexed: 30
Chunks in ChromaDB: 210

Resume indexing and verification passed.
```

```
sample_path = sorted(RESUME_DIR.glob("*.txt"))[0]

text = extract_text(sample_path)
metadata = extract_metadata(text, sample_path.name)

assert text.strip(), "Text extraction returned empty content."
assert metadata["name"] != "Unknown", "Name extraction failed."
assert metadata["skills"], "Skill extraction failed."

stored = rag.collection.get(include=["metadatas"])
indexed_names = {
    item["resume_filename"]
    for item in stored["metadatas"]
}

assert len(indexed_names) >= 30, (
    f"Expected 30 indexed resumes; found {len(indexed_names)}."
)

print("Text extraction through fs_tools: PASS")
print("Metadata extraction: PASS")
print("Unique resumes indexed:", len(indexed_names))
print("RAG regression test: PASS")
```

```
Text extraction through fs_tools: PASS
Metadata extraction: PASS
Unique resumes indexed: 30
RAG regression test: PASS
```

## 4. Hybrid Job Matching

The `JobMatcher` combines three ranking signals:

- **Semantic similarity:** captures contextual similarity between job descriptions and resume chunks.
- **TF-IDF lexical similarity:** measures keyword-based relevance.
- **Reciprocal Rank Fusion (RRF):** combines the rankings produced by semantic and lexical retrieval.

Candidate-level scores also incorporate required-skill coverage and experience. The resulting score is a heuristic ranking score from 0 to 100, not a calibrated probability of hiring success.

Eligibility is checked separately from ranking. Candidates missing must-have skills are ineligible. When a minimum experience requirement exists, candidates with insufficient or unknown experience are also ineligible. Ineligible candidates may still appear in the ranked results for transparency.

For each candidate, the system reports matched skills, missing must-have skills, experience, a reasoning summary, and an evidence excerpt.

```
class JobMatcher:
    """Hybrid resume retrieval and job-candidate matching."""

    def __init__(self, rag, top_k=10, rrf_k=60):
        self.rag = rag
        self.top_k = top_k
        self.rrf_k = rrf_k

    def parse_job_requirements(self, job_description):
        """Extract required skills, must-have skills, and experience."""
        text = job_description.strip()

        must_have_match = re.search(
```

```
            r"must[- ]have\s*:\s*(.+?)(?:\.|\n|$)",
            text,
            flags=re.IGNORECASE
        )

        must_have = []
        if must_have_match:
            raw_must_have = must_have_match.group(1)
            must_have = [
                skill.strip()
                for skill in raw_must_have.split(",")
                if skill.strip()
            ]

        found_skills = []
        for skill in SKILL_VOCABULARY:
            pattern = r"(?<!\w)" + re.escape(skill) + r"(?!\w)"
            if re.search(pattern, text, flags=re.IGNORECASE):
                found_skills.append(skill)

        experience_match = re.search(
            r"(?:minimum|min\.?)\s+experience\s*:\s*(\d+(?:\.\d+)?)\s*years?",
            text,
            flags=re.IGNORECASE
        )

        if not experience_match:
            experience_match = re.search(
                r"(\d+(?:\.\d+)?)\+?\s*years?",
                text,
                flags=re.IGNORECASE
            )

        minimum_experience = (
            float(experience_match.group(1))
            if experience_match
            else None
        )

        return {
            "required_skills": found_skills,
            "must_have_skills": must_have,
            "minimum_experience": minimum_experience
        }

    def match(self, job_description, top_k=None):
        """Rank resumes using semantic search, lexical search, and RRF."""
        started = time.perf_counter()

        top_k = top_k or self.top_k
        requirements = self.parse_job_requirements(job_description)

        stored = self.rag.collection.get(
            include=["documents", "metadatas"]
        )

        documents = stored["documents"]
        metadatas = stored["metadatas"]

        if not documents:
            raise ValueError("No indexed resume chunks found.")

        resume_chunks = defaultdict(list)

        for index, metadata in enumerate(metadatas):
            resume_chunks[metadata["resume_filename"]].append(index)

        semantic_hits = self.rag.search(
            job_description,
            top_k=len(documents)
        )

        semantic_rank = {}
        for rank, hit in enumerate(semantic_hits, start=1):
            key = (
                hit["metadata"]["resume_filename"],
                hit["metadata"]["chunk_index"]
            )
            semantic_rank[key] = rank

        vectorizer = TfidfVectorizer(
            lowercase=True,
```

```
    ngram_range=(1, 2),
    stop_words="english"
)

matrix = vectorizer.fit_transform(documents)
query_vector = vectorizer.transform([job_description])
lexical_scores = cosine_similarity(
    query_vector, matrix
).ravel()

lexical_order = lexical_scores.argsort()[::-1]
lexical_rank = {
    int(index): rank
    for rank, index in enumerate(lexical_order, start=1)
}

resume_rrf = defaultdict(float)
resume_semantic = defaultdict(float)
resume_lexical = defaultdict(float)
resume_best_chunk = {}

semantic_score_by_chunk = {}

for hit in semantic_hits:
    key = (
        hit["metadata"]["resume_filename"],
        hit["metadata"]["chunk_index"]
    )
    semantic_score_by_chunk[key] = hit["similarity"]

for index, metadata in enumerate(metadatas):
    filename = metadata["resume_filename"]
    chunk_key = (filename, metadata["chunk_index"])

    s_rank = semantic_rank.get(chunk_key)
    l_rank = lexical_rank.get(index)

    if s_rank is not None:
        resume_rrf[filename] += 1 / (self.rrf_k + s_rank)
        resume_semantic[filename] = max(
            resume_semantic[filename],
            semantic_score_by_chunk.get(chunk_key, 0.0)
        )

    if l_rank is not None:
        resume_rrf[filename] += 1 / (self.rrf_k + l_rank)
        resume_lexical[filename] = max(
            resume_lexical[filename],
            float(lexical_scores[index])
        )

    chunk_rrf = (
        (1 / (self.rrf_k + s_rank) if s_rank else 0)
        + (1 / (self.rrf_k + l_rank) if l_rank else 0)
    )

    if (
        filename not in resume_best_chunk
        or chunk_rrf > resume_best_chunk[filename]["rrf"]
    ):
        resume_best_chunk[filename] = {
            "rrf": chunk_rrf,
            "text": documents[index],
            "metadata": metadata
        }

candidates = []

for filename, indices in resume_chunks.items():
    metadata = metadatas[indices[0]]
    candidate_skills = [
        skill.strip()
        for skill in metadata.get("skills", "").split(",")
        if skill.strip()
    ]

    candidate_skills_lower = {
        skill.lower() for skill in candidate_skills
    }

    must_have = requirements["must_have_skills"]
    matched_must_have = [
```

```
    skill for skill in must_have
    if skill.lower() in candidate_skills_lower
]
missing_must_have = [
    skill for skill in must_have
    if skill.lower() not in candidate_skills_lower
]

years = metadata.get("experience_years", -1)
years = float(years) if years is not None else -1

minimum_experience = requirements["minimum_experience"]
experience_met = (
    None if minimum_experience is None or years < 0
    else years >= minimum_experience
)

rrf_value = resume_rrf[filename]
max_rrf = max(resume_rrf.values()) if resume_rrf else 1
normalized_rrf = rrf_value / max_rrf if max_rrf else 0

required_skills = requirements["required_skills"]
matched_required = [
    skill for skill in required_skills
    if skill.lower() in candidate_skills_lower
]
skill_coverage = (
    len(matched_required) / len(required_skills)
    if required_skills else 0
)

if minimum_experience is None:
    experience_score = 1.0
elif years < 0:
    experience_score = 0.0
else:
    experience_score = min(
        1.0, years / max(minimum_experience, 1)
    )

score = 100 * (
    0.60 * normalized_rrf
    + 0.25 * skill_coverage
    + 0.15 * experience_score
)

experience_eligible = (
    minimum_experience is None
    or (
        years >= 0
        and years >= minimum_experience
    )
)

eligible = (
    len(missing_must_have) == 0
    and experience_eligible
)
best = resume_best_chunk[filename]

candidates.append({
    "resume_filename": filename,
    "candidate_name": metadata.get(
        "candidate_name", "Unknown"
    ),
    "score": round(score, 2),
    "eligible": eligible,
    "skills": candidate_skills,
    "matched_skills": matched_required,
    "missing_must_have": missing_must_have,
    "experience_years": years if years >= 0 else None,
    "minimum_experience": minimum_experience,
    "experience_met": experience_met,
    "education": metadata.get(
        "education", "Not specified"
    ),
    "semantic_score": round(
        resume_semantic[filename], 4
    ),
    "lexical_score": round(
        resume_lexical[filename], 4
    ),
```

```
                "rrf_score": round(rrf_value, 6),
                "reasoning": (
                    f"Matched {len(matched_required)} of "
                    f"{len(required_skills)} required skills; "
                    f"matched {len(matched_must_have)} of "
                    f"{len(must_have)} must-have skills."
                ),
                "evidence_excerpt": best["text"][:500]
            })

        candidates.sort(
            key=lambda candidate: (
                candidate["eligible"],
                candidate["score"]
            ),
            reverse=True
        )

        elapsed = time.perf_counter() - started

        return {
            "requirements": requirements,
            "results": candidates[:top_k],
            "total_candidates": len(candidates),
            "latency_seconds": round(elapsed, 4)
        }


matcher = JobMatcher(rag, top_k=10)

print("JobMatcher initialized successfully.")
print("Hybrid retrieval: semantic + TF-IDF + RRF.")
print("Eligibility checks: must-have skills + minimum experience.")
```

```
JobMatcher initialized successfully.
Hybrid retrieval: semantic + TF-IDF + RRF.
Eligibility checks: must-have skills + minimum experience.
```

```
chunk_diagnostics = []

for resume_path in sorted(RESUME_DIR.iterdir()):
    if not resume_path.is_file():
        continue

    if resume_path.suffix.lower() not in {".txt", ".pdf", ".docx"}:
        continue

    text = extract_text(resume_path)

    for chunk in chunk_document(
        text,
        chunk_size=800,
        overlap=120
    ):
        chunk_diagnostics.append({
            "resume": resume_path.name,
            "section": chunk["section"],
            "characters": len(chunk["text"])
        })

chunk_df = pd.DataFrame(chunk_diagnostics)

print("Total chunks:", len(chunk_df))
print("\nChunk length statistics:")
display(chunk_df["characters"].describe().to_frame("characters"))

print("\nChunk lengths by section:")
display(
    chunk_df.groupby("section")["characters"]
    .agg(["count", "mean", "max"])
    .round(1)
    .sort_values("count", ascending=False)
)

print("\nChunks longer than each target:")
for size in [400, 800, 1200]:
    print(
        f"{size} characters: "
        f"{(chunk_df['characters'] > size).sum()} chunks exceed the target"
    )
```

```
Total chunks: 210

Chunk length statistics:
```

| | characters |
|---|---|
| count | 210.000000 |
| mean | 110.323810 |
| std | 65.284581 |
| min | 52.000000 |
| 25% | 65.000000 |
| 50% | 84.500000 |
| 75% | 126.000000 |
| max | 266.000000 |

```
Chunk lengths by section:
```

| section | count | mean | max |
|---|---|---|---|
| CERTIFICATIONS | 30 | 65.0 | 65 |
| EDUCATION | 30 | 56.0 | 56 |
| EXPERIENCE | 30 | 258.3 | 266 |
| GENERAL | 30 | 84.3 | 93 |
| PROJECTS | 30 | 113.5 | 126 |
| SUMMARY | 30 | 126.0 | 126 |
| TECHNICAL SKILLS | 30 | 69.2 | 84 |

```
Chunks longer than each target:
400 characters: 0 chunks exceed the target
800 characters: 0 chunks exceed the target
1200 characters: 0 chunks exceed the target
```

```
# rag = ResumeRAG(
#     db_dir=DB_DIR,
#     model_name="sentence-transformers/all-MiniLM-L6-v2"
# )

# rag.index_resumes(resume_dir=RESUME_DIR)

# # Create a fresh matcher connected to the restored collection
# matcher = JobMatcher(rag)

# # Verify the collection
# stored = rag.collection.get(include=["metadatas"])

# indexed_names = {
#     item["resume_filename"]
#     for item in stored["metadatas"]
# }

# print("Indexed resumes:", len(indexed_names))
# print("Indexed chunks:", len(stored["metadatas"]))

# assert len(indexed_names) >= 30
# print("Index and matcher restored.")
```

```
Loading embedding model...
```

Loading weights: 100% 103/103 [00:00<00:00, 2453.34it/s]

```
Embedding model loaded.
ChromaDB collection initialized.
Resumes indexed: 30
Chunks indexed: 210
Indexed resumes: 30
Indexed chunks: 210
Index and matcher restored.
```

```
job_files = sorted(JD_DIR.glob("*.txt"))

if len(job_files) < 5:
    raise ValueError(
```

```
        f"Expected at least 5 job descriptions; found {len(job_files)}."
    )

all_results = {}
summary_rows = []

for job_path in job_files:
    print("\n" + "=" * 80)
    print("JOB:", job_path.stem.replace("_", " ").title())
    print("=" * 80)

    job_description = extract_text(job_path)
    result = matcher.match(job_description, top_k=10)

    all_results[job_path.stem] = {
        "job_description": job_description,
        "requirements": result["requirements"],
        "total_candidates": result["total_candidates"],
        "latency_seconds": result["latency_seconds"],
        "results": result["results"]
    }

    print("Required skills:", result["requirements"]["required_skills"])
    print("Must-have skills:", result["requirements"]["must_have_skills"])
    print("Minimum experience:", result["requirements"]["minimum_experience"])
    print("Latency:", result["latency_seconds"], "seconds")

    display_df = pd.DataFrame([
        {
            "Rank": rank,
            "Candidate": candidate["candidate_name"],
            "Resume": candidate["resume_filename"],
            "Score": candidate["score"],
            "Eligible": candidate["eligible"],
            "Experience": candidate["experience_years"],
            "Matched Skills": ", ".join(candidate["matched_skills"]),
            "Missing Must-Haves": ", ".join(
                candidate["missing_must_have"]
            )
        }
        for rank, candidate in enumerate(result["results"], start=1)
    ])

    display(display_df)

    summary_rows.append({
        "job": job_path.stem,
        "candidate_count": result["total_candidates"],
        "returned_count": len(result["results"]),
        "eligible_count": sum(
            candidate["eligible"] for candidate in result["results"]
        ),
        "latency_seconds": result["latency_seconds"]
    })


results_path = RESULTS_DIR / "job_matching_results.json"

with results_path.open("w", encoding="utf-8") as file:
    json.dump(all_results, file, indent=2, ensure_ascii=False)

summary_df = pd.DataFrame(summary_rows)
summary_path = RESULTS_DIR / "job_matching_summary.csv"
summary_df.to_csv(summary_path, index=False)

print("\n" + "=" * 80)
print("RUN SUMMARY")
print("=" * 80)
display(summary_df)

print("JSON results saved to:", results_path)
print("CSV summary saved to:", summary_path)

assert len(all_results) >= 5
assert all(
    len(result["results"]) > 0
    for result in all_results.values()
)

print("\nAll job descriptions processed successfully.")
```

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | | | | | | | Terraform, Linux | |
| **4** | 5 | Finley Bose | candidate_19.txt | 93.74 | True | 3.0 | Docker, Kubernetes, AWS, Linux | |
| **5** | 6 | Drew Nair | candidate_09.txt | 93.10 | True | 5.0 | Docker, Kubernetes, AWS, Linux | |
| **6** | 7 | Avery Gupta | candidate_07.txt | 72.78 | False | 3.0 | AWS, Linux | Docker |
| **7** | 8 | Arden Chopra | candidate_27.txt | 72.18 | False | 5.0 | AWS, Linux | Docker |
| **8** | 9 | Hayden Reddy | candidate_17.txt | 71.21 | False | 7.0 | AWS, Linux | Docker |
| **9** | 10 | Dakota Sethi | candidate_21.txt | 70.69 | False | 5.0 | Docker, Terraform | AWS |

```
================================================================================
JOB: Data Engineer
================================================================================
Required skills: ['Python', 'PostgreSQL', 'SQL', 'Spark', 'Kafka']
Must-have skills: ['Python', 'SQL']
Minimum experience: 2.0
Latency: 0.5196 seconds
```

| | Rank | Candidate | Resume | Score | Eligible | Experience | Matched Skills | Missing Must-Haves |
|---|---|---|---|---|---|---|---|---|
| **0** | 1 | Sam Iyer | candidate_10.txt | 100.00 | True | 6.0 | Python, PostgreSQL, SQL, Spark, Kafka | |
| **1** | 2 | Remy Thomas | candidate_30.txt | 97.20 | True | 8.0 | Python, PostgreSQL, SQL, Spark, Kafka | |
| **2** | 3 | Emerson Pillai | candidate_20.txt | 96.68 | True | 4.0 | Python, PostgreSQL, SQL, Spark, Kafka | |
| **3** | 4 | Morgan Khan | candidate_04.txt | 73.98 | True | 6.0 | Python, SQL | |
| **4** | 5 | Reese Roy | candidate_14.txt | 73.52 | True | 4.0 | Python, SQL | |
| **5** | 6 | Kendall Dutta | candidate_24.txt | 72.97 | True | 8.0 | Python, SQL | |
| **6** | 7 | Alex Sharma | candidate_01.txt | 73.27 | False | 3.0 | Python, PostgreSQL | SQL |
| **7** | 8 | Dakota Sethi | candidate_21.txt | 69.76 | False | 5.0 | Python, PostgreSQL | SQL |
| **8** | 9 | Robin Kapoor | candidate_11.txt | 69.38 | False | 7.0 | Python, PostgreSQL | SQL |
| **9** | 10 | Avery Gupta | candidate_07.txt | 62.07 | False | 3.0 | Python | SQL |

```
================================================================================
JOB: Java Backend
================================================================================
Required skills: ['Java', 'Spring Boot', 'SQL']
```

Next steps: ~~Generate code with display_df~~ ~~New interactive sheet~~ ~~Generate code with display_df~~ New interactive sheet Generate co

```
import math
import pandas as pd

ground_truth = {
    "python_backend": {
        "candidate_01.txt", "candidate_11.txt", "candidate_21.txt"
    },
    "java_backend": {
        "candidate_02.txt", "candidate_12.txt", "candidate_22.txt"
    },
    "machine_learning": {
        "candidate_03.txt", "candidate_13.txt", "candidate_23.txt"
    },
    "cloud_devops": {
        "candidate_05.txt", "candidate_09.txt", "candidate_15.txt",
        "candidate_19.txt", "candidate_25.txt", "candidate_29.txt"
```

```
    },
    "data_engineer": {
        "candidate_10.txt", "candidate_20.txt", "candidate_30.txt"
    }
}

K = 10
evaluation_rows = []

for job_name, relevant in ground_truth.items():
    if job_name not in all_results:
        raise ValueError(f"Missing results for job: {job_name}")

    result = all_results[job_name]
    ranked = result["results"][:K]

    relevance = [
        int(candidate["resume_filename"] in relevant)
        for candidate in ranked
    ]

    hits = sum(relevance)
    precision_at_10 = hits / K
    recall_at_10 = hits / len(relevant) if relevant else 0.0
    reciprocal_rank = next(
        (1 / rank for rank, is_relevant in enumerate(relevance, start=1)
         if is_relevant),
        0.0
    )
    dcg = sum(
        rel / math.log2(rank + 1)
        for rank, rel in enumerate(relevance, start=1)
    )

    ideal_relevance = [1] * min(len(relevant), K)
    idcg = sum(
        rel / math.log2(rank + 1)
        for rank, rel in enumerate(ideal_relevance, start=1)
    )

    ndcg_at_10 = dcg / idcg if idcg else 0.0

    evaluation_rows.append({
        "job": job_name,
        "labeled_relevant_count": len(relevant),
        "relevant_retrieved_at_10": hits,
        "precision_at_10": precision_at_10,
        "recall_at_10": recall_at_10,
        "reciprocal_rank": reciprocal_rank,
        "ndcg_at_10": ndcg_at_10,
        "latency_seconds": result["latency_seconds"]
    })

evaluation_df = pd.DataFrame(evaluation_rows)

metric_columns = [
    "precision_at_10",
    "recall_at_10",
    "reciprocal_rank",
    "ndcg_at_10",
    "latency_seconds"
]

summary_metrics = (
    evaluation_df[metric_columns]
    .mean()
    .to_frame("mean")
    .reset_index(names="metric")
)

print("PER-JOB EVALUATION")
display(evaluation_df.round(4))

print("AGGREGATE METRICS")
display(summary_metrics.round(4))

evaluation_df.to_csv(
    RESULTS_DIR / "evaluation_metrics.csv",
    index=False
)

summary_metrics.to_csv(
    RESULTS_DIR / "evaluation_summary.csv",
```

```
    index=False
)

print("Saved evaluation_metrics.csv")
print("Saved evaluation_summary.csv")
```

```
PER-JOB EVALUATION
```

| | job | labeled_relevant_count | relevant_retrieved_at_10 | precision_at_10 | recall_at_10 | reciprocal_rank |
|---|---|---|---|---|---|---|
| **0** | python_backend | 3 | 3 | 0.3 | 1.0 | 1.0 |
| **1** | java_backend | 3 | 3 | 0.3 | 1.0 | 1.0 |
| **2** | machine_learning | 3 | 3 | 0.3 | 1.0 | 1.0 |
| **3** | cloud_devops | 6 | 6 | 0.6 | 1.0 | 1.0 |
| **4** | data_engineer | 3 | 3 | 0.3 | 1.0 | 1.0 |

```
AGGREGATE METRICS
```

| | metric | mean |
|---|---|---|
| **0** | precision_at_10 | 0.3600 |
| **1** | recall_at_10 | 1.0000 |
| **2** | reciprocal_rank | 1.0000 |
| **3** | ndcg_at_10 | 1.0000 |
| **4** | latency_seconds | 0.3774 |

```
Saved evaluation_metrics.csv
Saved evaluation_summary.csv
```

```
CHUNK_SIZES = [100, 200, 400]
experiment_rows = []

original_chunk_document = chunk_document

try:
    for size in CHUNK_SIZES:
        overlap = size // 5

        def experiment_chunk_document(
            text,
            chunk_size=size,
            overlap=overlap
        ):
            return original_chunk_document(
                text,
                chunk_size=chunk_size,
                overlap=overlap
            )

        chunk_document = experiment_chunk_document

        experiment_rag = ResumeRAG(
            db_dir=DB_DIR,
            model_name="sentence-transformers/all-MiniLM-L6-v2"
        )

        stats = experiment_rag.index_resumes(RESUME_DIR)
        experiment_matcher = JobMatcher(experiment_rag, top_k=10)

        for job_name, job_result in all_results.items():
            result = experiment_matcher.match(
                job_result["job_description"],
                top_k=10
            )

            relevant = ground_truth[job_name]
            ranked = result["results"][:10]

            relevance = [
                int(candidate["resume_filename"] in relevant)
                for candidate in ranked
            ]

            hits = sum(relevance)
            precision = hits / 10
            recall = hits / len(relevant) if relevant else 0.0

            reciprocal_rank = next(
```

```
                    (
                        1 / rank
                        for rank, rel in enumerate(relevance, start=1)
                        if rel
                    ),
                    0.0
                )

                dcg = sum(
                    rel / math.log2(rank + 1)
                    for rank, rel in enumerate(relevance, start=1)
                )

                ideal = [1] * min(len(relevant), 10)
                idcg = sum(
                    rel / math.log2(rank + 1)
                    for rank, rel in enumerate(ideal, start=1)
                )

                experiment_rows.append({
                    "chunk_size": size,
                    "job": job_name,
                    "chunks_indexed": stats["chunk_count"],
                    "precision_at_10": precision,
                    "recall_at_10": recall,
                    "reciprocal_rank": reciprocal_rank,
                    "ndcg_at_10": dcg / idcg if idcg else 0.0,
                    "latency_seconds": result["latency_seconds"]
                })

    finally:
        chunk_document = original_chunk_document


    experiment_df = pd.DataFrame(experiment_rows)

    experiment_summary = (
        experiment_df.groupby("chunk_size")
        .agg(
            chunks_indexed=("chunks_indexed", "first"),
            precision_at_10=("precision_at_10", "mean"),
            recall_at_10=("recall_at_10", "mean"),
            reciprocal_rank=("reciprocal_rank", "mean"),
            ndcg_at_10=("ndcg_at_10", "mean"),
            latency_seconds=("latency_seconds", "mean")
        )
        .reset_index()
        .round(4)
    )

    display(experiment_summary)

    experiment_df.to_csv(
        RESULTS_DIR / "chunk_size_experiment_details.csv",
        index=False
    )

    experiment_summary.to_csv(
        RESULTS_DIR / "chunk_size_experiment.csv",
        index=False
    )

    print("Updated chunk-size experiment saved.")
```

```
Loading embedding model...
```

Loading weights: 100% 103/103 [00:00<00:00, 3447.50it/s]

```
Embedding model loaded.
ChromaDB collection initialized.
Resumes indexed: 30
Chunks indexed: 360
Loading embedding model...
```

Loading weights: 100% 103/103 [00:00<00:00, 1906.71it/s]

```
Embedding model loaded.
ChromaDB collection initialized.
Resumes indexed: 30
Chunks indexed: 240
Loading embedding model...
```

Loading weights: 100% 103/103 [00:00<00:00, 2004.65it/s]

```
Embedding model loaded.
ChromaDB collection initialized.
Resumes indexed: 30
Chunks indexed: 210
```

| | chunk_size | chunks_indexed | precision_at_10 | recall_at_10 | reciprocal_rank | ndcg_at_10 | latency_seconds |
|---|---|---|---|---|---|---|---|
| 0 | 100 | 360 | 0.36 | 1.0 | 1.0 | 1.0 | 0.1088 |
| 1 | 200 | 240 | 0.36 | 1.0 | 1.0 | 1.0 | 0.1451 |
| 2 | 400 | 210 | 0.36 | 1.0 | 1.0 | 1.0 | 0.0930 |

```
Updated chunk-size experiment saved.
```

Next steps: Generate code with `experiment_summary` New interactive sheet

## 5. Evaluation Methodology

The system is evaluated using synthetic ground-truth labels based on the generated candidate profiles.

- **Precision@10:** fraction of the top 10 results labeled relevant.
- **Recall@10:** fraction of all labeled relevant resumes retrieved in the top 10.
- **Mean Reciprocal Rank (MRR):** rewards retrieving a relevant resume near the top of the ranking.
- **Normalized Discounted Cumulative Gain (nDCG@10):** measures the ranking quality of relevant results.
- **Latency:** elapsed time reported for each matching request.

The chunk-size experiment compares configurations of 100, 200, and 400 characters, with overlap set to 20% of the chunk size. The experiment records chunk counts, retrieval metrics, and latency.

Because the dataset is small and synthetic, these metrics demonstrate pipeline behavior rather than real-world hiring accuracy. Ground-truth labels are generated from the dataset design and are not independent human assessments.

```
test_results = []

for job_name, job_result in all_results.items():
    requirements = job_result["requirements"]
    candidates = job_result["results"]

    assert len(candidates) > 0, (
        f"{job_name}: no candidates returned"
    )

    assert len(candidates) <= 10, (
        f"{job_name}: more than 10 candidates returned"
    )

    for candidate in candidates:
        assert 0 <= candidate["score"] <= 100, (
            f"{job_name}: invalid score for "
            f"{candidate['resume_filename']}"
        )

    for candidate in candidates:
        if candidate["missing_must_have"]:
            assert candidate["eligible"] is False, (
                f"{job_name}: candidate with missing must-haves "
                "was marked eligible"
            )

    minimum_experience = requirements["minimum_experience"]

    if minimum_experience is not None:
        for candidate in candidates:
            years = candidate["experience_years"]
```

```
            if years is not None and years < minimum_experience:
                assert candidate["eligible"] is False, (
                    f"{job_name}: candidate with insufficient "
                    "experience was marked eligible"
                )

    test_results.append({
        "job": job_name,
        "status": "PASS",
        "candidates_returned": len(candidates),
        "eligible_returned": sum(
            candidate["eligible"] for candidate in candidates
        )
    })


test_df = pd.DataFrame(test_results)
display(test_df)

assert len(test_results) == 5
assert all(row["status"] == "PASS" for row in test_results)

print("All retrieval and eligibility checks passed.")
```

| | job | status | candidates_returned | eligible_returned |
|---|---|---|---|---|
| 0 | cloud_devops | PASS | 10 | 6 |
| 1 | data_engineer | PASS | 10 | 6 |
| 2 | java_backend | PASS | 10 | 3 |
| 3 | machine_learning | PASS | 10 | 3 |
| 4 | python_backend | PASS | 10 | 3 |

```
All retrieval and eligibility checks passed.
```

Next steps: Generate code with `test_df` New interactive sheet

```
import pandas as pd

edge_case_results = []

def record_test(name, passed, details=""):
    edge_case_results.append({
        "test": name,
        "status": "PASS" if passed else "FAIL",
        "details": details
    })

missing_skill_jd = """
Job Title: Python Backend Developer
Must-have: Python, Django.
Minimum Experience: 3 years.
Responsibilities: Build backend APIs and maintain services.
"""

try:
    response = matcher.match(missing_skill_jd, top_k=10)
    candidates = response["results"]

    missing_candidates = [
        c for c in candidates if c.get("missing_must_have")
    ]
    incorrectly_eligible = [
        c for c in missing_candidates if c.get("eligible") is True
    ]

    record_test(
        "Missing must-have skills",
        bool(missing_candidates) and not incorrectly_eligible,
        f"Missing-skill candidates: {len(missing_candidates)}; "
        f"incorrectly eligible: {len(incorrectly_eligible)}"
    )
except Exception as e:
    record_test("Missing must-have skills", False, str(e))

experience_jd = """
Job Title: Senior Developer
Must-have: Python.
```

```
Minimum Experience: 7 years.
Responsibilities: Develop and maintain software.
"""

try:
    response = matcher.match(experience_jd, top_k=10)
    candidates = response["results"]

    below_minimum = [
        c for c in candidates
        if c.get("experience_years") is not None
        and c["experience_years"] < 7
    ]
    incorrectly_eligible = [
        c for c in below_minimum if c.get("eligible") is True
    ]

    record_test(
        "Minimum experience filter",
        bool(below_minimum) and not incorrectly_eligible,
        f"Below minimum: {len(below_minimum)}; "
        f"incorrectly eligible: {len(incorrectly_eligible)}"
    )
except Exception as e:
    record_test("Minimum experience filter", False, str(e))

try:
    test_text = """
    Name: Test Candidate
    Skills: Python, SQL
    Education: B.Tech
    """
    metadata = extract_metadata(test_text, "test_resume.txt")

    record_test(
        "Missing experience metadata",
        metadata.get("experience_years") is None,
        f"Extracted experience: {metadata.get('experience_years')}"
    )
except Exception as e:
    record_test("Missing experience metadata", False, str(e))

no_skills_jd = """
Job Title: General Technology Professional
Responsibilities: Collaboration, problem solving, and documentation.
No specific technical requirements are listed.
"""

try:
    response = matcher.match(no_skills_jd, top_k=5)
    candidates = response["results"]

    record_test(
        "No recognized must-have skills",
        bool(candidates),
        f"Candidates returned: {len(candidates)}"
    )
except Exception as e:
    record_test("No recognized must-have skills", False, str(e))

try:
    records = list_files(str(RESUME_DIR))
    txt_records = [
        r for r in records if r.get("extension") == ".txt"
    ]
    read_result = (
        read_file(txt_records[0]["path"]) if txt_records else {}
    )

    record_test(
        "fs_tools file read",
        bool(txt_records) and read_result.get("success") is True,
        f"TXT files found: {len(txt_records)}"
    )
except Exception as e:
    record_test("fs_tools file read", False, str(e))

edge_case_df = pd.DataFrame(edge_case_results)
display(edge_case_df)

if (edge_case_df["status"] == "PASS").all():
    print("All edge-case tests passed.")
```